10 Graph feature engineering: Manual and semiautomated approaches

This chapter covers

- Manual feature engineering techniques for nodes and relationships in graphs
- Combining domain expertise with semiautomated extraction in a graph representation
- Real-world applications of feature engineering

The success of machine learning (ML) on graphs depends on a fundamental challenge: how to effectively represent graph elements (nodes, relationships, and entire graphs) as vectors that ML algorithms can process. This representation step, often called vectorization or featurization, determines how well our models can learn and make predictions.

Although modern ML algorithms—from traditional approaches like logistic regression and random forests to sophisticated deep learning models—are well-established, they can't directly process graph structures. Instead, they require numerical input vectors. The quality of these vectors directly affects the performance of any downstream task, whether it's classifying nodes, predicting relationships, or analyzing entire graphs.

This chapter explores the art and science of creating these vector representations, progressing from manual to automated approaches. We start with manual feature engineering, crafting interpretable features based on domain knowledge and graph properties. This hands-on approach, although time-consuming, provides insights into what makes representations effective and helps us understand why certain features work better than others.

We'll gradually introduce more automated techniques. However, the more automated our feature extraction becomes, the less interpretable the features tend to be. This creates a spectrum:

- *Manual features* are highly interpretable but labor-intensive to create (quickly introduced in chapter 9 and discussed in depth in this chapter).
- *Semiautomated features* strike a balance between interpretability and efficiency (also covered in this chapter).
- *Fully automated features* are efficient to generate but harder to interpret (discussed in chapters 11 and 12).

In traditional datasets, features are direct measurements from the real world. For example, a weather prediction dataset may include measurable attributes like precipitation, temperature range, and wind speed. During model training, we use rows where we know both the features (measured attributes) and the label (actual weather). For prediction, we apply our trained model to new data where we only have the features and need to determine the weather label.

But in graph-based ML, we must construct our features from the graph structure itself. Instead of physical measurements, we need to capture

meaningful properties of nodes, relationships, or entire graphs as numerical values.
There are two fundamental approaches to creating these graph-based features:

- *Feature engineering*, which we introduced in the previous chapter, relies on manually designed features based on graph properties and domain knowledge. These features are highly interpretable but time-consuming to create, and they may not capture all relevant patterns for complex tasks. Common examples of graph-based features include node degree, clustering coefficients, and centrality measures.
- *Representation learning*, in contrast, automatically learns feature representations from the graph structure. This approach requires minimal human intervention and can adapt to specific tasks through training. It often captures complex patterns more effectively than manual engineering, but it typically produces features that are harder to interpret. This approach will be covered in the next two chapters.

Understanding the challenges and limitations of feature engineering will help us appreciate why representation learning has become increasingly important. Manual feature engineering remains valuable for two key reasons. First, it produces interpretable features that humans can understand and validate. Second, it provides insights into what makes graph representations effective, informing the design of automated approaches.

Another significant advantage of manually extracted features is their compatibility with large language models (LLMs) for autonomous reasoning about graphs. Because these features are based on well-understood graph algorithms and properties, LLMs can effectively interpret and reason about them.

In this chapter, we'll explore three practical approaches to feature engineering, each drawn from real-world consulting projects. These examples demonstrate both the power and limitations of manual feature extraction in graph-based ML.

10.1 Manual node features

Suppose we have a network of people: that is, a graph with nodes that represent people and relationships that represent any connection among these individuals (see figure 10.1). Among these people are some recognized fraudsters. We don't have a full list of them, so our work is to classify nodes to recognize the unrecognized fraudsters or determine the risk of people being victims of fraudulent activities.

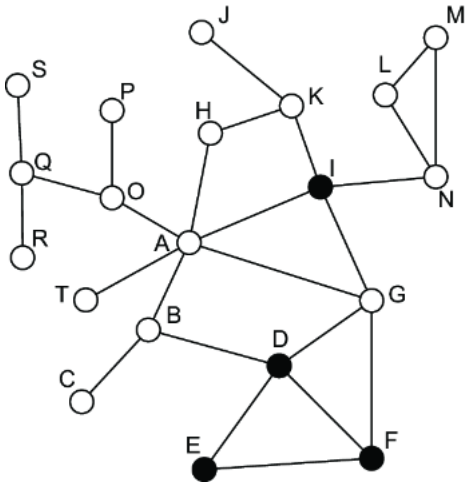

Figure 10.1 An example social network in which white nodes represent legitimate individuals or those whose status as fraudsters is unknown, and black nodes represent known fraudsters

The network includes two types of nodes: black nodes represent known fraudsters (nodes D, E, F, and I), and white nodes represent legitimate users or unidentified nodes. All connections between nodes are represented as undirected edges. The following listing shows how to create this network using Python and NetworkX.

Listing 10.1 Creating the fraud detection network example

```
import networkx as nx
import matplotlib.pyplot as plt

def create_fraud_network():
    G = nx.Graph()   #1

    fraudsters = ['D', 'E', 'F', 'I']   #2

    # Add all nodes first
    nodes = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J',
             'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T']   #3

    G.add_nodes_from(nodes)

    for node in G.nodes():
        G.nodes[node]['is_fraudster'] = node in fraudsters   #4

    edges = [
            ('A', 'B'), ('A', 'G'), ('A', 'H'), ('A', 'I'), ('A', 'O'),
            ('A', 'T'), ('B', 'D'), ('B', 'C'), ('D', 'E'), ('D', 'F'),
            ('D', 'G'), ('E', 'F'), ('F', 'G'), ('G', 'I'), ('H', 'K'),
            ('I', 'K'), ('I', 'N'), ('K', 'J'), ('L', 'M'), ('L', 'N'),
            ('N', 'M'), ('O', 'P'), ('O', 'Q'), ('Q', 'R'), ('Q', 'S')
    ]   #5

    G.add_edges_from(edges)

    return G   #6
```

#1 Initializes the empty undirected graph
#2 Fraudulent nodes (black in figure 10.1)
#3 Defines all nodes in the network
#4 Sets the fraudster attribute for each node, based on the list
#5 Defines all edges in the network
#6 Returns the complete graph

You can use this network for all the subsequent analyses we perform in the chapter by using the following call.

Listing 10.2 Example use

```
G = create_fraud_network()   #1

degree_metrics = compute_degree_metrics(G)  #2
triangle_metrics = compute_triangle_metrics(G)  #2
```

#1 You can use this graph object with any of our metric calculations.
#2 Example metrics that we define later

Our goal is to implement a classifier that, given a node, will say whether it is a fraudster. Generally, this task is accomplished by using well-known classical classification algorithms, such as logistic regression, Bayesian classifier, decision tree, random forest, and so on. They are all supervised algorithms, so during training, the input is a set of features associated with each data point (nodes, in this case) and a label (here, fraudster/not fraudster). After the training builds a ML model, the classifier will be able to assign labels with a certain likelihood.

To use such algorithms, we have to "represent" each node as a set of features that can be used during training and prediction as good indicators, as illustrated in figure 10.2. We can extract a lot of interesting information for each node:

- *Local features* are those we can extract considering the node's one-hop neighborhood or *ego-centered network* (*egonet*)—that is, a particular node and its immediate neighbors. The center of the egonet is the *ego*, and the surrounding nodes are the *alters*. These local features can also consider an *n*-order neighborhood around the ego node, considering nodes that are *n* hops from that node.
- *Global features* measure the role of each node in the entire network or a great portion of it (not in the egonet or an *n*-order neighborhood). In this category fall centrality metrics like betweenness centrality, closeness centrality, PageRank, and Eigenvector centrality. These measures capture a node's influence in the network and how the node can be influenced by others. (If you are not familiar with these centrality algorithms, we recommend our earlier book, *Graph-Powered Machine Learning* [1], or [2] as a reference.)

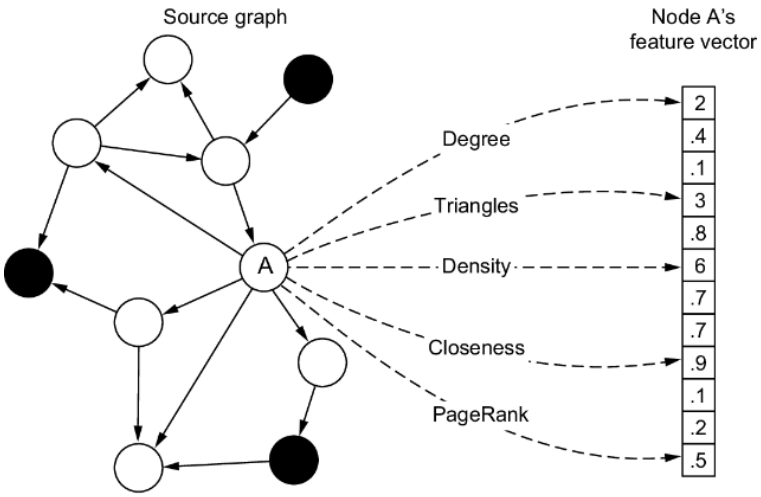

Figure 10.2 Node feature extraction: using metrics and graph algorithms to transform nodes into numerical feature vectors that capture key characteristics

We'll use metrics to identify features representing each node. We'll also customize some to improve the classification's final quality and demonstrate how the featurization process can be tailored to our needs. Our approach will progress from local to global metrics, starting with features based on immediate neighbors before examining network-wide patterns. In each case, we'll define the metric and its significance, present code for automated extraction, and display the results in a table.

We'll build these features incrementally, showing how each new metric adds a dimension to our node representations. This systematic approach allows us to capture increasingly complex patterns of network structure and node behavior.

10.1.1 Degree

The degree of a node represents how many neighbors the node has. In our example case, we want to distinguish between the number of fraudulent and legitimate direct neighbors. These are called *fraudulent* and *legitimate degrees* (which we shorten to fraud degree and legit degree). These two measures, together with the global degree, provide a better representation of the node's direct connections. For example, if I have 10 direct neighbors who are all fraudulent, the chances that I am a fraudster are higher than if all my neighbors were legit. The following listing computes these values in a generic graph.

Listing 10.3 Computing types of node degrees in a fraud detection network

```
def compute_degree_metrics(G):
    degree_metrics = {}   #1

    for node in G.nodes():
        neighbors = list(G.neighbors(node))
        total_degree = len(neighbors)  #2

        fraud_degree = sum(1 for neighbor in neighbors
                           if G.nodes[neighbor].get('is_fraudster', False))  #3

        legit_degree = total_degree - fraud_degree   #4

        degree_metrics[node] = {
            'total_degree': total_degree,
            'fraud_degree': fraud_degree,
            'legit_degree': legit_degree
        }

    return degree_metrics  #5

def get_node_degrees(G, node):
    metrics = compute_degree_metrics(G)
    return metrics.get(node, {
        'total_degree': 0,
        'fraud_degree': 0,
        'legit_degree': 0
    })   #6
```

#1 Initializes a dictionary to store degree metrics for each node
#2 Calculates the total number of neighbors (total degree)
#3 Counts neighbors marked as fraudsters using the node attribute is_fraudster
#4 Calculates the legit degree as total minus fraudulent neighbors
#5 Returns a dictionary containing all degree metrics for each node
#6 Gets degree metrics for a specific node

This code assumes that the graph has nodes marked with an '`is_fraudster`' boolean attribute to identify fraudulent nodes. Table 10.1 contains

the related values. For example, node D has a total of four direct neighbors, of which two are fraudulent and two are legitimate.

Table 10.1 Global degree, fraud degree, and legit degree of the fraud graph in figure 10.1

Node	A	B	C	D	E	F	G	H	I	J
Total degree	6	3	1	4	2	3	4	2	4	1
Fraud degree	1	1	0	2	2	2	3	0	0	0
Legit degree	5	2	1	2	0	1	1	2	4	1
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
Total degree	3	2	2	3	3	1	3	1	1	1
Fraud degree	1	0	0	1	0	0	0	0	0	0
Legit degree	2	2	2	2	3	1	3	1	1	1

EXERCISE

Compute the values yourself manually, and check them to verify that the concepts are clear. These are very simple measures to compute; later, the others will require to be run by code.

10.1.2 Triangles

In graph theory, a *triangle* is a subgraph that consists of three nodes which are all connected. So, if we have three nodes—A, B, and C—they form a triangle if relationships exist between the couples A-B, A-C, and B-C (see figure 10.3).

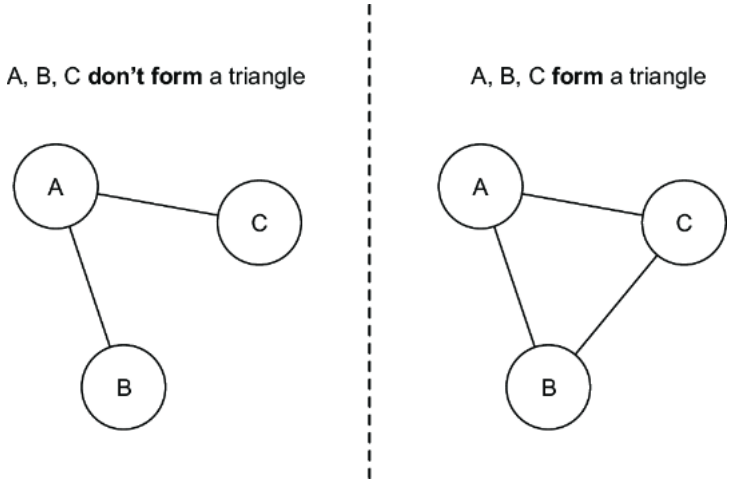

Figure 10.3 Examples of three connected nodes: if each node is connected to the other two, they constitute a triangle.

The presence of triangles in a node's egonet is an indication that the target node has strong connections with its neighbors. Think about the people close to you: your friends are probably also friends with each other. That's why triangles reveal the influential effect of a closely connected group of individuals. Consider a node and two alters forming a triangle. In our example, if both alters are fraudulent, we can conclude that the triangle is fraudulent, and vice versa. If only one alter is fraudulent, the triangle is called *semifraudulent*.

The following code computes the total number of triangles and the fraudulent, legitimate, and semifraudulent triangles in our graph.

Listing 10.4 Computing triangle metrics in a fraud detection network

```
import networkx as nx

def compute_triangle_metrics(G):
    triangle_metrics = {}   #1

    for node in G.nodes():
        triangles = []
        neighbors = list(G.neighbors(node))

        for i in range(len(neighbors)):
            for j in range(i + 1, len(neighbors)):
                if G.has_edge(neighbors[i], neighbors[j]):  #2
                    triangles.append((neighbors[i], neighbors[j]))

        total_triangles = len(triangles)   #3
        fraud_triangles = 0   #3
        legit_triangles = 0   #3
        semi_fraud_triangles = 0   #3

        for n1, n2 in triangles:   #4
            n1_fraud = G.nodes[n1].get('is_fraudster', False)
            n2_fraud = G.nodes[n2].get('is_fraudster', False)

            if n1_fraud and n2_fraud:   #5
                fraud_triangles += 1
            elif not n1_fraud and not n2_fraud:  #6
                legit_triangles += 1
            else:   #7
                semi_fraud_triangles += 1

        triangle_metrics[node] = {
            'total_triangles': total_triangles,
            'fraud_triangles': fraud_triangles,
            'legit_triangles': legit_triangles,
            'semi_fraud_triangles': semi_fraud_triangles
        }

    return triangle_metrics   #8

def get_node_triangles(G, node):
    metrics = compute_triangle_metrics(G)
    return metrics.get(node, {
        'total_triangles': 0,
        'fraud_triangles': 0,
        'legit_triangles': 0,
        'semi_fraud_triangles': 0
    })   #9
```

#1 Initializes a dictionary to store triangle metrics for each node
#2 Finds triangles by checking whether two neighbors of the target node are connected
#3 Initializes counts
#4 Classifies each triangle based on the fraud status of the two other nodes
#5 Counts a triangle as fraudulent if both other nodes are fraudsters
#6 Counts a triangle as legit if both other nodes are legit
#7 Counts a triangle as semifraudulent if one other node is fraudulent and one is legit
#8 Returns a dictionary containing all triangle metrics for each node
#9 Gets triangle metrics for a specific node

Table 10.2 contains the values for our fraudulent graph.

Table 10.2 The triangle measures of the fraud graph in figure 10.1

Node	A	B	C	D	E	F	G	H	I	J
Total triangles	1	0	0	2	1	2	2	0	1	0
Fraud triangles	0	0	0	1	1	1	1	0	0	0
Legit triangles	0	0	0	0	0	0	0	0	1	0
Semifraud triangles	1	0	0	1	0	1	1	0	0	0
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
Total triangles	0	1	1	1	0	0	0	0	0	0
Fraud triangles	0	0	0	0	0	0	0	0	0	0
Legit triangles	0	1	1	1	0	0	0	0	0	0
Semifraud triangles	0	0	0	0	0	0	0	0	0	0

EXERCISE

Again, compute the values yourself, and check them to be sure the concepts are clear.

10.1.3 Density

Density is another measure that indicates how nodes can influence each other. It measures the extent to which nodes in a graph are connected. Given a fully connected graph of *N* nodes, the following formula computes the total number of possible edges:

$$\binom{N}{2} = \frac{N(N-1)}{2}$$

In this case, each node is connected to every other node in the network. The density measures the portion of these possible edges that are observed in the actual graph. So, if *M* is the number of edges in the graph, the density of the entire network can be computed with the following formula:

$$d = \frac{M}{\binom{N}{2}} = \frac{2M}{N(N-1)}$$

We can also compute the density for each node considering the density of its egonet. For example, suppose node A is the target node. Its egonet has 7 nodes, so the total number of possible edges in the egonet is 7(7 – 1)/2 = 21. The number of observed edges is 7, and d = 7/21 = ~0.33. In this example, the calculation is simple; the next listing contains the code for when things get more difficult. This measure, due to its nature, does not include specific values related to the fraud domain we are considering (there is no fraud density, or legit density).

Listing 10.5 Computing egonet density in a fraud detection network

```
import networkx as nx

def compute_density_metrics(G):
    density_metrics = {}   #1

    for node in G.nodes():
        neighbors = list(G.neighbors(node))
        egonet_nodes = neighbors + [node]   #2


        N = len(egonet_nodes)   #3

        if N < 2:  # Handle special case where egonet is too small
            density_metrics[node] = 0.0
            continue

        M = 0
        for i in range(len(egonet_nodes)):
            for j in range(i + 1, len(egonet_nodes)):
                if G.has_edge(egonet_nodes[i], egonet_nodes[j]):   #4
                    M += 1

        max_possible_edges = (N * (N - 1)) / 2   #5
        density = M / max_possible_edges #6

        density_metrics[node] = round(density, 2)   #7

    return density_metrics   #8

def get_node_density(G, node):
    metrics = compute_density_metrics(G)
    return metrics.get(node, 0.0)   #9
```

#1 Initializes a dictionary to store density values for each node
#2 Creates an egonet by getting the node's neighbors plus the node itself
#3 Gets the number of nodes in the egonet
#4 Counts the actual number of edges (M) in the egonet
#5 Calculates maximum possible edges N(N – 1)/2 in the egonet
#6 Calculates the density as ratio of actual edges to maximum possible edges
#7 Rounds the density to two decimal places
#8 Returns the dictionary containing density values for all nodes
#9 Gets the density value for a specific node

Running this listing on our example graph gives the values in table 10.3.

Table 10.3 Density measures of the fraud graph in figure 10.1

Node	A	B	C	D	E	F	G	H	I	J
Density	0.33	0.5	1	0.6	1	0.83	0.6	0.66	0.5	1
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
Density	0.5	1	1	0.66	0.5	1	0.5	1	1	1

The measures we have seen so far use the egonet of a node to compute values. The measures that follow consider the entire network during computation.

10.1.4 Geodesic (or shortest) path

The *geodesic path* or *shortest path* represents the minimum distance between two nodes. We can use this measure and customize it to the specif-

ic needs of our domain to identify features that could be the input for downstream algorithms.
In our example, we want to know whether there are paths between fraudulent nodes and legitimate nodes, how long they are, and how many of these the network contains. If more paths exist between two nodes, and those paths are short, there is a higher chance that fraudulent behavior will affect the target node. Based on these considerations, we decide to take the shortest path to a fraudulent node (the geodesic path). We also need to know the number of fraudulent nodes surrounding a node at a certain distance, so we consider the number of paths with one, two, or three hops to any fraudulent node.

This time, before showing the code, let's start with some examples that can be computed manually. Node A is connected to node I via a single hop (direct connection). The geodesic distance is, hence, 1. There are no other direct connections to other fraudulent nodes: the number of one-hop paths is one. There are four two-hop paths—A-G-I, A-B-D, A-G-D, and A-G-F—and so on; see table 10.4.

Table 10.4 Geodesic paths of the fraud graph in figure 10.1

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**
Geodesic paths	1	1	2	0	0	0	1	2	0	2
1-hop paths	1	1	0	2	2	2	3	0	0	0
2-hop paths	4	3	1	8	4	7	5	2	6	1
3-hop paths	18	13	3	19	15	17	25	4	9	0
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
Geodesic paths	1	2	2	1	2	3	3	4	4	2
1-hop paths	1	0	0	1	0	0	0	0	0	0
2-hop paths	0	1	1	0	1	0	0	0	0	1
3-hop paths	9	1	1	8	4	1	1	0	0	4

Our network is straightforward, and computing even the three-hop paths is not complicated. But for real networks, manual computation is infeasible and not practical.

Calculating geodesic paths is computationally expensive. Various algorithms are available, and one of the best for our case is Dijkstra's.

NOTE *Dijkstra's algorithm* finds the shortest path between nodes in a graph by iteratively selecting the unvisited node with the smallest tentative distance, calculating distances through it to each unvisited neighbor, and marking the node as visited. The algorithm efficiently builds up the shortest path tree one vertex at a time [3].

Listing 10.6 contains the code to automatically extract the features related to the geodesic distance. It uses the library `networkx`, which has an implementation of the Dijkstra algorithm from a single source. Even Neo4j,

in the current GDS library (https://github.com/neo4j/graph-data-science), has a couple of Dijkstra implementations, one of which computes the shortest paths between a source node and all nodes reachable from that node.

Listing 10.6 Computing geodesic path metrics in a fraud detection network

```
import networkx as nx
from collections import defaultdict

def compute_geodesic_metrics(G, max_hops=3):
    path_metrics = {}   #1

    fraudster_nodes = [n for n, attr in G.nodes(data=True)
                       if attr.get('is_fraudster', False)]   #2

    for node in G.nodes():
        if G.nodes[node].get('is_fraudster', False):
            geodesic_path = 0   #3
            hop_counts = defaultdict(int)
        else:
            paths_to_fraudsters = []   #4
            hop_counts = defaultdict(int)

            for fraudster in fraudster_nodes:
                try:
                    path = nx.shortest_path(G, node, fraudster)  #5
                    path_length = len(path) - 1
                    paths_to_fraudsters.append(path_length)

                    if path_length <= max_hops:
                        hop_counts[path_length] += 1   #6
                except nx.NetworkXNoPath:
                    continue

            geodesic_path = min(paths_to_fraudsters)
              if paths_to_fraudsters else float('inf')   #7

        path_metrics[node] = {
            'geodesic_path': geodesic_path,
            '#1-hop_paths': hop_counts[1],
            '#2-hop_paths': hop_counts[2],
            '#3-hop_paths': hop_counts[3]
        }   #8

    return path_metrics   #9

def get_node_paths(G, node):
    metrics = compute_geodesic_metrics(G)
    return metrics.get(node, {
        'geodesic_path': float('inf'),
        '#1-hop_paths': 0,
        '#2-hop_paths': 0,
        '#3-hop_paths': 0
    })   #10
```

#1 Initializes a dictionary to store path metrics for each node

#2 Identifies all fraudulent nodes in the graph

#3 If the node is fraudulent, the distance to the nearest fraudster is 0.

#4 For nonfraudulent nodes, calculates paths to all fraudsters

#5 Uses Dijkstra's algorithm (via networkx) to find the shortest path to each fraudster

#6 Counts the number of paths for each hop distance up to max_hops

#7 Finds the shortest path length to any fraudster

#8 Stores metrics, including shortest path and count of paths at each hop distance

#9 Returns a dictionary containing all path metrics for each node

#10 Gets path metrics for a specific node

This implementation computes the geodesic distance from the starting node to fraudster nodes only and up to a predefined distance.

10.1.5 Closeness

Closeness centrality represents how "close" a node is to all other nodes. It measures the mean distance from a node to all other nodes in the network, where the distance between nodes is computed as the geodesic or shortest path (described in the previous section) between them. Given a network with N nodes, the mean geodesic distance or "farness" from node i to the other nodes is computed as follows:

$$g(v_i) = \frac{\sum_{j=1\ (j \neq i)}^{N} d(v_i,\ v_j)}{N-1}$$

Let's examine each part:

- The numerator $\sum_{j=1,\ j \neq i}^{N} d(v_i, v_j)$ adds up all the shortest path distances:
  - $d(v_i, v_j)$ represents the shortest path distance between node v_i and another node, v_j.
  - The summation goes from $j = 1$ to N, covering all nodes in the network. $j \neq i$ in the sum indicates that we skip the distance to the node itself (which would be 0).
- The denominator ($N - 1$) represents the following:
  - N is the total number of nodes in the network.
  - We subtract 1 because we don't include the distance from a node to itself.
  - This gives us the number of other nodes in the network.

Thus this formula calculates the average by adding up all the shortest paths from node v_i to every other node and dividing by the number of other nodes. A lower $g(v_i)$ value indicates that the node is generally closer to other nodes in the network, whereas a higher value suggests it tends to be farther away.

For example, if a node has many direct connections, its shortest paths to other nodes will tend to be small, resulting in a lower $g(v_i)$ value. This indicates the node is well-connected and centrally positioned in the network. In the fraud use case, when a fraudulent node has a low value for $g(v_i)$, fraud may spread easily through the network and contaminate other nodes faster.

Closeness centrality is the inverse of farness because we assign higher values to nodes that are more central in the network. The formula is as follows:

$$\text{closeness centrality}(v_i) = \left(\frac{\sum_{j=1\ (j \neq i)}^{N} d(v_i,\ v_j)}{N-1} \right)^{-1}$$

Two problems can arise. First, the values of the closeness centralities for all the nodes in the network may be close together, so we often have to look at the decimal places to see the differences. Second, when a node cannot reach another (no path exists between the two), the distance between them is infinite. To overcome this problem, closeness centrality excludes the distances to nodes that cannot be reached. So, when the closeness centrality is computed for a node, the calculation doesn't use the full network—it only uses the portion of the network reachable from that node.

The following code computes the closeness centrality for a network represented using a `networkx` graph.

Listing 10.7 Computing closeness centrality in a fraud detection network

```
import networkx as nx
from collections import defaultdict

def compute_closeness_metrics(G):
    closeness_metrics = {}   #1

    for node in G.nodes():
        total_distance = 0
        reachable_nodes = 0

        shortest_paths = nx.single_source_shortest_path_length(G, node)   #2

        for other_node, distance in shortest_paths.items():
            if other_node != node:   #3
                total_distance += distance
                reachable_nodes += 1   #4

        n = len(G.nodes()) - 1  #5
        if reachable_nodes > 0 and n > 0:
            closeness = (reachable_nodes / n) * (reachable_nodes /
                total_distance)   #6
        else:
            closeness = 0.0

        closeness_metrics[node] = round(closeness, 2)   #7

    return closeness_metrics   #8

def get_node_closeness(G, node):
    metrics = compute_closeness_metrics(G)
    return metrics.get(node, 0.0)  #9

def analyze_closeness_distribution(G):
    metrics = compute_closeness_metrics(G)
    values = list(metrics.values())

    stats = {
        'max_closeness': max(values),
        'min_closeness': min(values),
        'avg_closeness': sum(values) / len(values),
        'most_central_node': max(metrics.items(), key=lambda x: x[1])[0],
        'least_central_node': min(metrics.items(), key=lambda x: x[1])[0]
    }   #10

    return stats
```

#1 Initializes a dictionary to store closeness values for each node
#2 Uses networkx to calculate shortest paths to all nodes
#3 Excludes self
#4 Counts reachable nodes and sums up the distances
#5 Total nodes minus self
#6 Calculates the normalized closeness centrality considering disconnected components
#7 Rounds the closeness to two decimal places
#8 Returns a dictionary containing closeness values for all nodes
#9 Gets closeness value for a specific node
#10 Analyzes the distribution of closeness values

The code first determines the shortest paths from a target node to all other nodes in the network using `networkx`'s path-finding algorithms. This step gives us the fundamental distances needed for the centrality calculation.

However, real-world networks may not be fully connected. To handle this common scenario, the code incorporates a normalization strategy that

considers only the reachable nodes. This approach prevents disconnected components from distorting the centrality values while still providing meaningful measurements for partially connected networks.
The computation of closeness centrality follows the formula we introduced earlier. To make the results more practical and interpretable, we've included analytical capabilities that help understand the distribution of closeness values across the network. This can be particularly valuable when trying to identify nodes that serve as central points in the network's structure.

By processing each node this way, we obtain a comprehensive view of how central each node is relative to the rest of the network, with values normalized between 0 and 1 for easier comparison and analysis (see table 10.5). This implementation strikes a balance between theoretical correctness and practical applicability, making it suitable for both research and real-world applications.

Table 10.5 Closeness metrics of the fraud graph in figure 10.1

Node	A	B	C	D	E	F	G	H	I	J
Closeness	0.5	0.39	0.28	0.35	0.26	0.33	0.43	0.37	0.45	0.26
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
Closeness	0.34	0.26	0.26	0.34	0.40	0.29	0.31	0.24	0.24	0.34

Node A is the most closely connected to all other nodes in the network. Nodes R and S are the farthest away from all other nodes.

10.1.6 Betweenness

Betweenness centrality helps us understand a node's importance in a network from a different perspective than closeness centrality. Whereas closeness measures how quickly a node can reach others, betweenness measures how often a node acts as a bridge between other nodes. Specifically, it quantifies the number of times a node appears on the shortest paths between other pairs of nodes.

For any pair of nodes in a network, information or influence likely flows along the shortest path between them. If a particular node frequently appears on these shortest paths, it has high betweenness centrality and thus potentially controls the flow of information among many other nodes. Mathematically, the betweenness centrality of a node v is calculated as

$$\text{betweenness}(v) = \sum_{s,t\ (s \neq t \neq v)} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} represents the total number of shortest paths between nodes s and t, and $\sigma_{st}(v)$ represents the number of those paths that pass through node v. As shown in the next listing, this sum is taken over all pairs of nodes s and t where neither is equal to v.

Listing 10.8 Computing betweenness centrality in a fraud detection network

```
import networkx as nx
from collections import defaultdict

def compute_betweenness_metrics(G, normalized=True):
    betweenness_metrics = {}   #1

    betweenness = nx.betweenness_centrality(
        G,
        normalized=normalized,
        endpoints=False   #2
    )

    for node in G.nodes():
        betweenness_metrics[node] = round(betweenness[node], 3)  #3

    return betweenness_metrics  #4

def analyze_betweenness_distribution(G):
    metrics = compute_betweenness_metrics(G)
    values = list(metrics.values())

    return {
        'max_betweenness': max(values),
        'min_betweenness': min(values),
        'avg_betweenness': sum(values) / len(values),
        'key_bridges': [node for node, score in metrics.items()
                        if score > sum(values) / len(values)]  #5
    }

def get_node_betweenness(G, node):
    metrics = compute_betweenness_metrics(G)
    return metrics.get(node, 0.0)   #6

def identify_potential_bottlenecks(G, threshold=0.5):
    metrics = compute_betweenness_metrics(G)

    bottlenecks = {node: score for node, score in metrics.items()
                   if score > threshold}   #7
    return bottlenecks
```

#1 Initializes a dictionary to store betweenness values for all nodes
#2 Calculates betweenness using networkx's implementation
#3 Rounds values to three decimal places for readability and stores them
#4 Returns a dictionary containing betweenness values for all nodes
#5 Identifies nodes with above-average betweenness as key bridges
#6 Gets the betweenness value for a specific node
#7 Identifies potential bottlenecks based on a threshold

The code uses `networkx`'s betweenness centrality algorithm while adding practical functionality for analysis and interpretation. The computed betweenness values are normalized by default, meaning they are scaled to fall between 0 and 1, making them easier to compare across different networks. A value closer to 1 indicates that the node appears on many shortest paths and thus has a high potential for controlling information flow.

Looking at the results in table 10.6, we see interesting patterns. Node A, with a betweenness centrality of 104, appears to be a crucial bridge in the network, potentially controlling the flow of information among many other nodes. In contrast, nodes C, E, J, L, M, P, R, S, and T all have betweenness values of 0, indicating that they don't act as bridges between any pairs of nodes.

The implementation includes additional analytical tools that help identify potential bottlenecks and key bridges in the network. These can be useful when trying to understand vulnerabilities in the network structure or identify nodes that might require additional monitoring in a fraud detection context.

Table 10.6 Betweenness metrics of the fraud graph in figure 10.1

Node	A	B	C	D	E	F	G	H	I	J
Betweenness	104	24.67	0	13.83	0	6.167	35.3	9	65	0
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
Betweenness	20	0	0	34	63	0	35	0	0	0

10.1.7 PageRank

PageRank is a powerful metric that measures node importance based on the structure of incoming connections. Originally developed by Google's founders to rank web pages, it has proven valuable in many other network analysis contexts, including fraud detection. Unlike simpler centrality measures, PageRank considers not just the quantity of connections but also their quality: a node connected to other highly ranked nodes will receive a higher PageRank score.

In our fraud detection context, we can adapt PageRank in two interesting ways. First, we'll compute a base PageRank that considers all connections equally. Then we'll compute a fraud-weighted PageRank where connections from known fraudulent nodes carry more weight. This dual approach, shown in the following listing, will help us understand both a node's general importance in the network and its specific relationship to fraudulent activity [4].

Listing 10.9 Computing PageRank variations for fraud detection

```
import networkx as nx
import numpy as np

def compute_pagerank_metrics(G, fraud_weight=2.0, damping_factor=0.85):
    pagerank_metrics = {}   #1

    base_pagerank = nx.pagerank(
        G,
        alpha=damping_factor,
        personalization=None,
        weight=None
    )   #2

    fraud_personalization = {}
    for node in G.nodes():   #3
        if G.nodes[node].get('is_fraudster', False):
            fraud_personalization[node] = fraud_weight
        else:
            fraud_personalization[node] = 1.0

    fraud_pagerank = nx.pagerank(
        G,
        alpha=damping_factor,
        personalization=fraud_personalization,
        weight=None
    )  #4

    for node in G.nodes():
        pagerank_metrics[node] = {
            'pagerank_base': round(base_pagerank[node], 3),
            'pagerank_fraud': round(fraud_pagerank[node], 3)
        }   #5

    return pagerank_metrics   #6


def get_node_pagerank(G, node):
    metrics = compute_pagerank_metrics(G)
    return metrics.get(node, {
        'pagerank_base': 0.0,
        'pagerank_fraud': 0.0
    })   #7
```

#1 Initializes a dictionary to store both PageRank variations
#2 Calculates standard PageRank using networkx implementation
#3 Creates a personalization dictionary that gives higher weight to fraudulent nodes
#4 Calculates fraud-weighted PageRank using personalization
#5 Stores both PageRank values for each node
#6 Returns a dictionary containing all PageRank metrics
#7 Gets the PageRank values for a specific node

The results for each node in our example graph are shown in table 10.7.

Table 10.7 PageRank metrics of the fraud graph in figure 10.1

Node	A	B	C	D	E	F	G	H	I	J
PageRank **base**	0.108	0.057	0.023	0.068	0.036	0.051	0.067	0.04	0.07	0.024
PageRank **fraud**	0.087	0.063	0.018	0.168	0.114	0.145	0.109	0.023	0.094	0.011
Node	**K**	**L**	**M**	**N**	**0**	**P**	**Q**	**R**	**S**	**T**
PageRank **base**	0.06	0.041	0.041	0.057	0.066	0.026	0.75	0.028	0.028	0.022
PageRank **fraud**	0.039	0.016	0.016	0.034	0.02	0.005	0.011	0.003	0.003	0.012

Node A has the highest base PageRank (0.108), indicating its general importance in the network structure. However, when we look at the fraud-weighted PageRank, node D emerges as the most significant (0.168), suggesting that it has stronger connections to fraudulent activity despite not having the highest base PageRank.

This divergence between base PageRank and fraud-weighted PageRank can provide valuable insights. Nodes that show a significant increase in their fraud-weighted PageRank compared to their base PageRank might warrant closer investigation, as they have more substantial connections to known fraudulent nodes than their overall network position suggests.

10.1.8 Prediction

We could continue extracting features, but at this point it should be clear how the process works. The process should be iterative; features can change and increase until the classifier reaches the quality of prediction you feel is enough. In this section, we will use the features extracted so far and perform a prediction using the same algorithm (logistic regression) we used in chapter 9. The following listing extracts a vector of features for each node and prepares the dataset for the next step.

Listing 10.10 Creating node features

```
import pandas as pd
import numpy as np
def create_node_features_dataset(G):
    degree_metrics = compute_degree_metrics(G)
    triangle_metrics = compute_triangle_metrics(G)
    density_metrics = compute_density_metrics(G)
    path_metrics = compute_geodesic_metrics(G)
    closeness_metrics = compute_closeness_metrics(G)
    betweenness_metrics = compute_betweenness_metrics(G)
    pagerank_metrics = compute_pagerank_metrics(G)   #1

    features_dict = {}
    for node in G.nodes():
        features_dict[node] = {
            'total_degree': degree_metrics[node]['total_degree'],
            'fraud_degree': degree_metrics[node]['fraud_degree'],
            'legit_degree': degree_metrics[node]['legit_degree'],

            'total_triangles': triangle_metrics[node]['total_triangles'],
            'fraud_triangles': triangle_metrics[node]['fraud_triangles'],
            'legit_triangles': triangle_metrics[node]['legit_triangles'],
            'semi_fraud_triangles': triangle_metrics[node]['semi_fraud_triangles'],

            'density': density_metrics[node],
            'geodesic_path': path_metrics[node]['geodesic_path'],
            'paths_1hop': path_metrics[node]['#1-hop_paths'],
            'paths_2hop': path_metrics[node]['#2-hop_paths'],
            'paths_3hop': path_metrics[node]['#3-hop_paths'],
            'closeness': closeness_metrics[node],
            'betweenness': betweenness_metrics[node],
            'pagerank_base': pagerank_metrics[node]['pagerank_base'],
            'pagerank_fraud': pagerank_metrics[node]['pagerank_fraud'],

            # Label
            'is_fraudster': G.nodes[node]['is_fraudster']
        }   #2

    # Convert to DataFrame
    df = pd.DataFrame.from_dict(features_dict, orient='index')   #3

    return df
```

#1 Computes all metrics for each node

#2 Creates a comprehensive feature set for each node

#3 Converts to a pandas DataFrame for easier manipulation

Now that we have our features for each node in a pandas DataFrame, we can split them and use part during training and part to validate the quality of the trained model, as shown in the following listing.

Listing 10.11 Training a fraud classifier and evaluating its accuracy

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

def train_fraud_classifier(G):
    df = create_node_features_dataset(G)

    X = df.drop('is_fraudster', axis=1)
    y = df['is_fraudster']   #1

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )  #2

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)  #3

    clf = LogisticRegression(random_state=42, max_iter=1000)
    clf.fit(X_train_scaled, y_train)   #4

    y_pred = clf.predict(X_test_scaled)   #5

    feature_importance = pd.DataFrame({
        'feature': X.columns,
        'importance': abs(clf.coef_[0])
    }).sort_values('importance', ascending=False)   #6

    results = {
        'classification_report': classification_report(y_test, y_pred),
        'confusion_matrix': confusion_matrix(y_test, y_pred),
        'feature_importance': feature_importance,
        'model': clf,
        'scaler': scaler
    }   #7

    return results
```

#1 Separates features (X) from labels (y)
#2 Splits the data into training (80%) and testing (20%) sets
#3 Scales the features to normalize their ranges
#4 Trains the logistic regression classifier
#5 Calculates feature importance
#6 Collects all evaluation metrics
#7 Makes predictions

The stratified split (obtained by setting the parameter `stratify` to `yes`) ensures that both training and testing sets maintain the same proportion of fraudulent and legitimate nodes as the original dataset. `StandardScaler` ensures that all features are on the same scale, which is important for logistic regression. The feature importance analysis helps us understand which metrics are most useful for detecting fraudulent nodes.

With the model in our hands, we are now ready to predict the chance of a node being a fraudster by using the following function.

Listing 10.12 Function to predict the class of a node using a training model

```
def predict_fraud_probability(G, node, trained_results):
    df = create_node_features_dataset(G)
    node_features = df.loc[node].drop('is_fraudster')   #1

    scaled_features = trained_results['scaler'].transform(
        node_features.values.reshape(1, -1)
    )   #2

    fraud_prob =
        trained_results['model'].predict_proba(scaled_features)[0][1]  #3

    return fraud_prob
```

#1 Gets node features
#2 Scales features
#3 Gets the probability

All the functions are in place. Now we can run a simple analysis and a test to reveal the five most important features that indicate whether a node represents a fraudster or a legitimate person.

Listing 10.13 Obtaining the results of the entire process

```
G = create_fraud_network()
results = train_fraud_classifier(G)   #1

print("Classification Report:")
print(results['classification_report'])   #2

print("\nConfusion Matrix:")
print(results['confusion_matrix'])   #3

print("\nTop 5 Most Important Features:")
print(results['feature_importance'].head()) #4

node_of_interest = 'A'
prob = predict_fraud_probability(G, node_of_interest, results)
print(f"\nFraud probability for node {node_of_interest}: {prob:.3f}")   #5
```

#1 Creates and analyzes the network
#2 Prints a classification report
#3 Prints a confusion matrix
#4 Shows the top five most important features
#5 Example: Gets the fraud probability for a specific node

We will not look at the results, as they were obtained from a sample graph that was too small to be statistically relevant. But this process can also be applied to larger, more realistic graphs. An advantage of this approach is that even though the process was complex and required extensive feature design and careful consideration, it is fully under your control, and each extracted feature is easy to explain just by looking at the graph. This increases the transparency of the process and is why this method is valid when, for example, the database has a limited size or when explainability is important.

10.2 Manual relationship features

After exploring how to represent nodes through structural features, we now turn our attention to another fundamental challenge in graph ML: how to represent relationships between nodes. Although nodes are the primary elements in a graph, the connections between them often hold information that we need to capture and analyze.

Imagine a scenario in which we want to predict potential interactions between elements in a graph. This is where relationship prediction (also known as link prediction) comes into play. Whether we're trying to predict if two proteins might interact, if a customer might be interested in a product, or if a drug could treat a disease, we're essentially asking the same question: given two nodes, how likely is it that a relationship exists between them?

Similar to node classification, relationship prediction requires us to transform graph elements into features that ML algorithms can process. However, instead of representing individual nodes, we need to capture the characteristics of potential connections between nodes. This can be approached as a binary classification task (predicting whether a relationship exists or not) or as a multiclass classification task (predicting the type of relationship that might exist).

To demonstrate these concepts, we'll explore a practical application in drug repurposing: finding new uses for existing drugs. This task can be modeled as a link prediction problem between drugs (also referred more formally as compounds) and diseases, where we aim to predict potential therapeutic relationships. Through this example, we'll examine different strategies for creating meaningful representations of relationships in graphs, building on the concepts we explored for node feature engineering. Figure 10.4, taken from chapter 9, compares the two tasks—node classification and link prediction—and anticipates how the representation of two nodes can be combined to obtain the representation of a relationship.

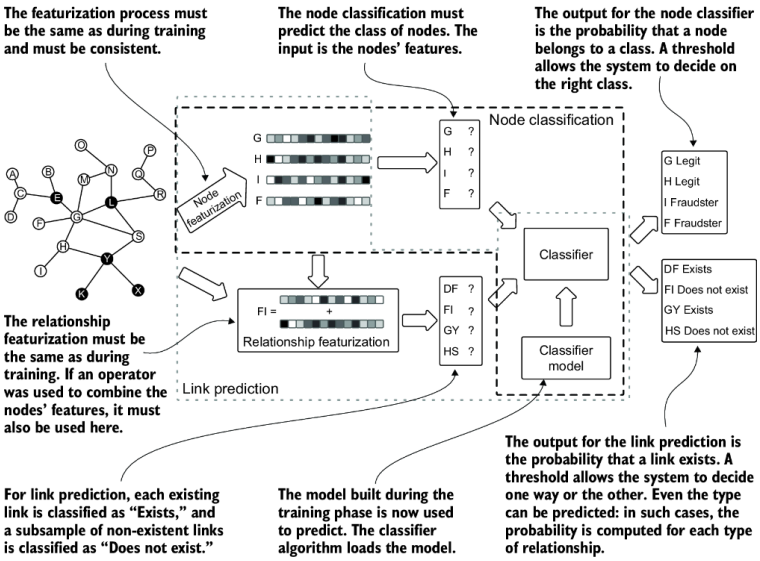

Figure 10.4 A typical relationship prediction, compared with the node classification process

The key difference in node classification is that we use a node pair as input to indicate the link we want to predict or use during training. Thus for the link prediction to be accurate, we need an effective way to represent the possible connections between the node pair used as input. We can use two distinct approaches:

- *Node-based combination*—We derive the relationship representation by combining the feature vectors of the source and target nodes. For example, if we have node representations [1,2,3] and [4,5,6], we might combine them through operations like concatenation or element-wise multiplication to represent their potential connection. This is the case presented in figure 10.4.
- *Path-based features*—Instead of relying on node features, we characterize relationships by analyzing the ways nodes are connected in the

graph. Each feature represents a distinct path pattern between nodes, such as the number of two-hop paths or the presence of specific meta-paths, creating a vector that captures the structural context of the relationship.

Node-based combinations work well with node embeddings, whereas path-based features excel at capturing complex network patterns. Let's examine each in detail.

10.2.1 Node-based representation

The most straightforward approach is to combine the feature representations of the two nodes that may be connected. Each node has a set of features that describe it (like a fingerprint), and we want to merge these features to describe what a connection between them might look like. This combination should work for any pair of nodes, whether they're connected in our graph or not. This is important because when we're predicting links, we need to evaluate both existing and potential connections.

The following are the most common techniques used to combine vectors with link prediction in mind:

- *Catenate*—Joins two vectors end to end. For vectors u and v of length n, it creates a new vector of length $2n$ by placing the elements of u followed by the elements of v. For example, if u = [1,2] and v = [3,4], their concatenation is [1,2,3,4]. This preserves all original information but doubles the dimension of the resulting vector.
- *Average*—Creates a new vector by taking the element-wise average (mean) of the two input vectors. For each position i, it computes $(u[i] + v[i])/2$. This maintains the original dimension while capturing the central tendency between the two vectors. For example, if u = [2,4] and v = [4,8], their average is [3,6].
- *L1* (*Manhattan distance*)—Computes the absolute difference between corresponding elements of the two vectors. For each position i, it calculates $|u[i] - v[i]|$. This captures how different the vectors are at each dimension and is useful for measuring dissimilarity. For example, if u = [1,4] and v = [3,1], their L1 combination is [2,3].
- *L2* (*Euclidean distance*)—Computes the squared difference between corresponding elements. For each position i, it calculates $(u[i] - v[i])^2$. Like L1, it captures differences between vectors but emphasizes larger differences due to the squaring operation. For example, if u = [1,4] and v = [3,1], their L2 combination is [4,9].
- *Hadamard* (*element-wise product*)—Multiplies the corresponding elements of the two vectors. For each position i, it computes $u[i] \times v[i]$. This operation is particularly useful when values in both vectors represent related quantities that should be combined multiplicatively. For example, if u = [2,4] and v = [3,1], their Hadamard product is [6,4].

The following listing shows how to implement each of these methods.

Listing 10.14 Combining two node representations into one link representation

```
def catenate(u, v):
    return u + v

def operator_avg(u, v):
    return (u + v) / 2.0

def operator_l1(u, v):
    return np.abs(u - v)

def operator_l2(u, v):
    return (u - v) ** 2

def operator_hadamard(u, v):
    return u * v
```

The quality of the link prediction depends directly on the quality of the nodes' representation. The composition technique also influences the final quality. There is no generic rule to help you with the selection, so a benchmark can give you an indication of which is the right approach for your scenario.

10.2.2 Path-based features

Some techniques describe the node pair representation, considering the source and destination nodes but not using their representation (or not using them exclusively). Many of these techniques require manual work to properly represent the relationships.

The manual process of representation is generally domain-specific, so it requires some understanding of the domain and the goal we want to achieve. As an example, like in chapter 4, we'll consider biomedical knowledge graphs and how they can help in drug discovery or repurposing. We will again use the Hetionet dataset, which integrates information from 19 public databases containing more than 50,000 nodes representing drugs, diseases, genes, and symptoms.

NOTE If you are following along with all the examples, you should already have the database. If not, go back to chapter 4 to see how to create it.

Himmelstein et al. [5] made significant advances in drug repurposing using the Hetionet dataset. Through computational analysis, they established more than 2 million relationships among these elements. And their work yielded practical results: they identified existing drugs used for depression and alcoholism that showed potential for treating smoking addiction and epilepsy. Let's examine their methodological approach at a high level.

Consider the Hetionet schema presented in chapter 4, repeated here in figure 10.5. The goal is to train a ML model to translate the network connectivity of a compound and a disease into a probability of treatment [6, 7]. To create a representation of each node pair—only compounds and diseases—that reflects the network connectivity between the nodes, the researchers evaluated all the metapaths that traverse from compound to disease and have lengths of 2 to 4. Figure 10.6 is a reminder of the concepts of metagraphs and metapaths, using examples between genes and diseases.

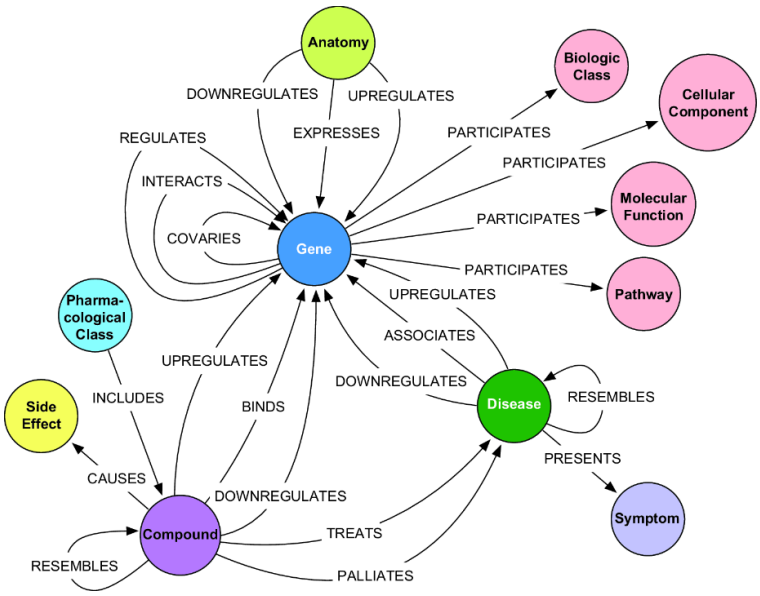

Figure 10.5 The Hetionet schema as it was presented in chapter 4

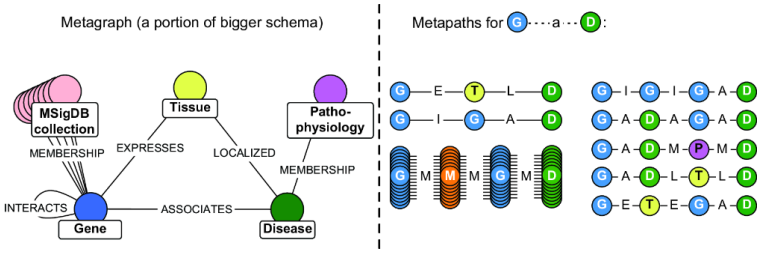

Figure 10.6 Metagraph and metapath examples from Hetionet

Starting from the schema in figure 10.5, some of the metapaths between *Compound* and *Disease* are listed in table 10.8. These metapaths represent just a subset of possible paths between compounds and diseases. They form features for each compound–disease pair, regardless of whether a direct connection exists.

Table 10.8 Examples of metapaths connecting drugs to diseases [5]

Metapath	Length	Abbr.
Compound–binds– *Gene*–associates– *Disease*	2	CbGaD
Compound–downregulates– *Gene*–upregulates– *Disease*	2	CdGuD
Compound–resembles– *Compound*–treats– *Disease*	2	CrCtD
Compound–binds– *Gene*–binds– *Compound*–treats– *Disease*	3	CbGbCtD
Compound–binds– *Gene*–expresses– *Anatomy*–localizes– *Disease*	3	CbGeAlD
Compound–binds– *Gene*–interacts– *Gene*–interacts– *Gene*–associates– *Disease*	4	CbGiGiGaD
Compound–binds– *Gene*–participates– *Pathway*–participates– *Gene* –associates– *Disease*	4	CbGpPWpGaD

For example, considering the compound *metformin* and the disease *type 2 diabetes*, we need to compute the value for each metapath (CbGaD, CdGuD, CrCtD, etc.); see figure 10.7. The simplest approach would be to count distinct path instances between the compound and disease. However, simply counting paths can be misleading if nodes with very high connectivity dominate the counts. For example, if a gene is involved in many biological processes, it will naturally appear in more paths, but this doesn't necessarily indicate stronger or more meaningful relationships between compounds and diseases.

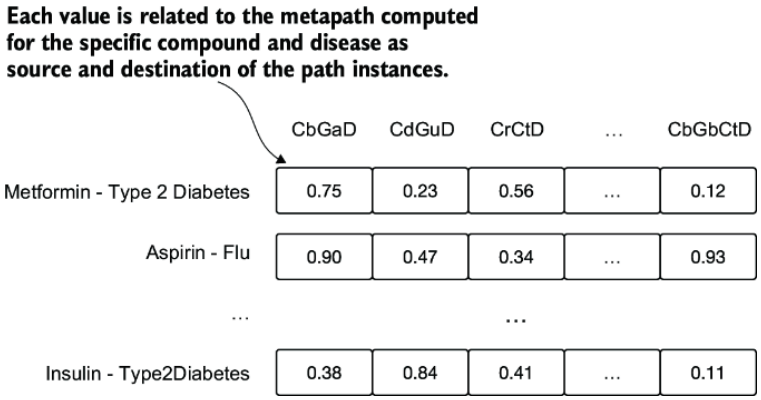

Figure 10.7 Metapath-based feature definition for the relationships between drugs and diseases

To address this bias, we use the degree-weighted path count (DWPC; discussed in more detail in chapter 4), which applies a damping factor based on node degrees. When we calculate DWPC,

- Each path is weighted inversely to the degrees of intermediate nodes
- Nodes with many connections contribute less to the final score
- The damping effect helps highlight more specific, focused biological pathways.

For instance, suppose we have two paths between metformin and type 2 diabetes:

- One through a highly connected gene (degree 100)
- Another through a more specific gene (degree 10)

The second path will contribute more significantly to the DWPC score, better reflecting potential biological significance. This approach has proven particularly effective in drug repurposing, as it helps identify meaningful therapeutic relationships while reducing noise from hub nodes in the biological network.

The next listing computes the DWPC between metformin and type 2 diabetes for the CbGaD (Compound–binds–Gene–associates–Disease) metapath using Neo4j.

Listing 10.15 DWPC between metformin and type 2 diabetes for CbGaD

```
MATCH path = (c:Compound)-[:BINDS_CbG]-(g)-[:ASSOCIATES_DaG]-(d:Disease)
WHERE c.name = 'Metformin' AND d.name = 'type 2 diabetes mellitus'
WITH
[
  count{(v)-[:BINDS_CbG]-()},
  count{()-[:BINDS_CbG]-(g)},
  count{(g)-[:ASSOCIATES_DaG]-()},
  count{()-[:ASSOCIATES_DaG]-(d)}
]
AS degrees, path, d
WITH
  d.identifier AS disease_id,
  d.name AS disease_name,
  count(path) AS PC,
  sum(reduce(pdp = 1.0, d in degrees| pdp * d ^ -0.4)) AS DWPC
RETURN
  disease_id, disease_name, PC, DWPC
```

Running it on our Hetionet database, we get the value 0.0007. We will use this value for the feature CbGaD in the vector for the *Metformin–Type 2 Diabetes* pair.

EXERCISE

Run the query in listing 10.15 for other compounds and diseases, considering existing and non-existent direct connections. Then change the query to test other metapaths.

Listings 10.16 and 10.17 provide two more examples.

Listing 10.16 DWPC value for CbGeAlD

```
MATCH path = (c:Compound)-[:BINDS_CbG]-(g:Gene)<-[:EXPRESSES_AeG]-
[CA](a:Anatomy)<-[:LOCALIZES_DlA]-(d:Disease)
WHERE c.name = 'Metformin' AND d.name = 'type 2 diabetes mellitus'
WITH
[
  count{(c)-[:BINDS_CbG]-()},
  count{()-[:BINDS_CbG]-(g)},
  count{(g)<-[:EXPRESSES_AeG]-()},
  count{()-[:EXPRESSES_AeG]-(a)},
  count{(a)<-[:LOCALIZES_DlA]-()},
  count{()-[:LOCALIZES_DlA]-(d)}
] AS degrees, path, d
WITH
  d.identifier AS disease_id,
  d.name AS disease_name,
  count(path) AS PC,
  sum(reduce(pdp = 1.0, d in degrees| pdp * d ^ -0.4)) AS DWPC
RETURN disease_id, disease_name, PC, DWPC
```

Listing 10.17 DWPC value for CbGpPWpGaD

```
MATCH path = (c:Compound)-[:BINDS_CbG]-(g1:Gene)-[:PARTICIPATES_GpPW]->(pw:Pathway)
WHERE c.name = 'Metformin' AND d.name = 'type 2 diabetes mellitus'
WITH
[
  count{(c)-[:BINDS_CbG]-()},
  count{()-[:BINDS_CbG]-(g1)},
  count{(g1)-[:PARTICIPATES_GpPW]->()},
  count{()-[:PARTICIPATES_GpPW]->(pw)},
  count{(pw)<-[:PARTICIPATES_GpPW]-()},
  count{()<-[:PARTICIPATES_GpPW]-(g2)},
  count{(g2)-[:ASSOCIATES_DaG]-()},
  count{()-[:ASSOCIATES_DaG]-(d)}
] AS degrees, path, d
WITH
  d.identifier AS disease_id,
  d.name AS disease_name,
  count(path) AS PC,
  sum(reduce(pdp = 1.0, d in degrees| pdp * d ^ -0.4)) AS DWPC
RETURN disease_id, disease_name, PC, DWPC
```

Computing the DWPC values for all possible compound–disease pairs across all potential metapaths would be computationally expensive and could lead to noisy or misleading results. To address this challenge, Himmelstein and colleagues [5] developed a two-step approach to reduce complexity and eliminate features that might be irrelevant or potentially harmful to the model's performance:

1. *Metapath reduction*—They developed a statistical method to identify the most significant metapaths by analyzing their frequency in known treatment-versus-nontreatment relationships. This analysis reduced the number of relevant metapaths from 1,026 to 709 while maintaining high predictive power.
2. *Pair selection*—They further refined the analysis by using domain knowledge and degree-based probabilistic analysis to identify the most promising compound–disease pairs. This reduction not only decreased computational overhead but also improved classifier performance by focusing on the most relevant pairs.

Although combining node representations (as discussed earlier) offers a simpler approach, it often performs poorly with manually extracted node features. As we'll explore later, this approach becomes more effective when working with automatically extracted features.

USING LLMS FOR GRAPH FEATURE ENGINEERING

LLMs, as we have seen many times throughout the book, can provide support in complex tasks. Graph feature engineering is one of them. LLMs excel at tasks that require understanding complex patterns and translating them into executable code. This is especially valuable when working with graph databases, where crafting queries often requires a deep understanding of both the domain and the query language. For instance, in our drug repurposing case, LLMs can do the following:

- *Query generation*—LLMs can translate high-level descriptions of metapaths into optimized Cypher queries.
- *Feature engineering*—They can suggest relevant patterns and relationships that might not be immediately obvious.
- *Code generation*—They can help create the necessary infrastructure to execute and process query results.

For example, suppose we want to generate Cypher queries for multiple metapaths in our drug repurposing network to extract features for our relationships representation. Here's an effective prompt we can use with an LLM:

You are a graph database expert specializing in Neo4j and Cypher queries. I'm working on a drug repurposing project and need help generating queries for metapath analysis.

I'll provide you with:

- The graph schema (obtained from apoc.meta.schema())
- An example of the query for CbGaD
- A list of metapaths between Compound and Disease nodes
- Sample compound and disease names for testing

For each metapath:

- Generate a Cypher query that computes both Path Count (PC) and Degree-Weighted Path Count (DWPC, using damping factor 0.4)
- Include degree calculations for each node in the path
- Return disease_id, disease_name, PC, and DWPC

The schema is:

{Shema definition here or as attachment}

An example of the query for DWPC is:

```
MATCH path = (c:Compound)-[:BINDS_CbG]-(g)-[:ASSOCIATES_DaG]-
             (d:Disease)
WHERE c.name = 'Metformin' AND d.name = 'type 2 diabetes mellitus'
WITH
[
  count{(v)-[:BINDS_CbG]-()},
  count{()-[:BINDS_CbG]-(g)},
  count{(g)-[:ASSOCIATES_DaG]-()},
  count{()-[:ASSOCIATES_DaG]-(d)}
]
AS degrees, path, d
WITH
  d.identifier AS disease_id,
  d.name AS disease_name,
  count(path) AS PC,
  sum(reduce(pdp = 1.0, d in degrees| pdp * d ^ -0.4)) AS DWPC
RETURN
  disease_id, disease_name, PC, DWPC
```

Please generate queries for these metapaths:

- CbGaD (Compound-binds-Gene-associates-Disease)
- CdGuD (Compound-downregulates-Gene-upregulates-Disease)
- Use {source compound} as the compound and {destination disease} as the disease.

This is an example of a prompt you can use to generate Cypher queries; then you can write the Python code to generate the final representation. You can change the prompt to return results that are immediately useful, but our goal was to plant the seed for how you can use LLMs creatively.

10.3 Semiautomated feature extraction

In sections 10.1 and 10.2, we explored manual feature engineering for both nodes and relationships, demonstrating how domain knowledge can guide the creation of meaningful representations. We saw how careful selection of structural metrics and domain-specific patterns can create effective features for tasks like fraud detection and drug repurposing. But although this manual approach provides deep insights, it comes with significant challenges: it requires extensive domain expertise, is time-consuming to implement, and needs to be customized for each new use case.

What if we could maintain the benefits of manual feature engineering—interpretability, reliability, predictability—while automating much of the feature selection process? This is where ReFeX (Recursive Feature eXtraction) [8] comes in, offering a middle ground between fully manual feature engineering and the complex neural network approaches we'll explore in chapters 11 and 12. ReFeX automatically identifies and extracts relevant structural features from graphs. Unlike black-box neural approaches, ReFeX's process is transparent and produces interpretable features that domain experts can understand and validate. This transparency is particularly valuable when we need to explain why certain predictions were made—imagine a fraud detection system where we need to justify why an account was flagged as suspicious.

Another advantage of ReFeX is its consistency across different graphs. The features it generates can be compared meaningfully between different networks or even different snapshots of the same network over time—something that is rarely possible with more complex neural approaches. This property makes ReFeX particularly valuable for applications where we need to track how graph structures evolve or compare patterns across different networks.

To demonstrate these benefits, we'll first examine how ReFeX works by manually calculating its features on a small example. Then we'll see how it can be applied to larger, real-world scenarios where manual feature engineering would be impractical.

ReFeX works by recursively generating local and egonet features from the graph structure, automatically capturing many of the metrics we previously calculated manually. The key advantages of this approach are as follows:

- *Efficiency*—Automated extraction of recursive structural features
- *Consistency*—Systematic approach to feature generation
- *Interpretability*—Generated features that maintain clear structural meaning
- *Scalability*—Ability to handle larger graphs while maintaining feature quality

However, human oversight remains essential in this process. Domain experts can

- *Validate* the relevance of generated features
- *Incorporate* domain knowledge to guide feature selection
- *Understand* and explain the features' contribution to fraud detection
- *Modify* the feature extraction process based on specific requirements

This hybrid approach sets the stage for our later discussion of fully automated feature learning methods, where we'll see how deep learning techniques can extract features without human intervention, trading interpretability for potentially more sophisticated pattern recognition.

ReFeX computes features by recursively combining local (node-based) features with features coming from the node's neighborhood (egonet-based). In this way, the algorithm produces the regional features—capturing "behavioral" information—that represent the kind of nodes to which a given node is connected, as opposed to the identity of those nodes. It is who you know, or who you relate to, that matters in mining across different graphs.

The ReFeX process is based on two fundamental rules:

- *Structural*—The construction of feature matrix F should not require additional attribute information on nodes or links.
- *Effective*—Good node features should (1) help us predict node attributes when such attributes are available and (2) be transferable across graphs (e.g., when the graph changes over time).

The ideal feature set should help with data mining tasks. Typical tasks include node classification (after we are given some labels), de-anonymization of the graph nodes, and transfer learning. Figure 10.8 represents, in a very simple way, the input and the output of the process.

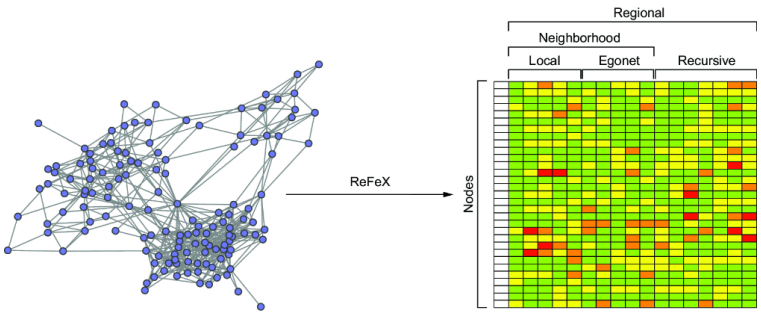

Figure 10.8 ReFeX converts each node into a vector representing the node's topological feature at different scales [8].

ReFeX operates on the pure structural aspects of the graph—nodes and relationships—without considering node labels or types. This focus on topology allows the algorithm to identify structural patterns. The feature extraction process occurs in three main stages:

1. *Local feature extraction* focuses on immediate node characteristics. The primary metric at this level is the node degree, including both weighted and unweighted variants. When working with directed graphs, ReFeX computes both in-degree and out-degree separately. For weighted graphs, it calculates the weighted degree as the sum of incident edge weights, providing a more nuanced view of node connectivity.
2. *Examination of egonet features* analyzes each node's immediate neighborhood. At this level, ReFeX computes metrics including the number of incoming egonet edges, outgoing egonet edges, and total egonet edges. When working with weighted graphs, it also calculates weighted variants of these metrics to capture the strength of connections within the ego network.
3. *Recursive feature extraction* aggregates existing features through the recursive application of summary statistics. This process uses combinations of aggregation functions (sum/mean) to capture increasingly complex structural patterns. For example, a feature like degree(sum)(mean)(mean)(sum) can capture regional structural patterns that extend beyond the immediate neighborhood. In directed graphs, ReFeX computes these recursive features separately for incoming and outgoing paths, providing a comprehensive view of directional patterns in the network.

The recursive nature of the algorithm can generate an exponentially growing number of features. To manage this complexity, ReFeX uses several pruning techniques:

- *Correlation analysis*—Identifies and eliminates highly correlated feature pairs
- *Logarithmic binning*—Maps feature values to discrete intervals for efficient comparison
- *Threshold-based pruning*—Removes features that differ by less than a specified threshold

10.3.1 Performing ReFeX manually

Let's apply ReFeX manually to the small graph database shown in figure 10.9. We are considering an undirected and unweighted graph for simplicity, and we won't perform any pruning steps. Table 10.9 shows the degree of each node.

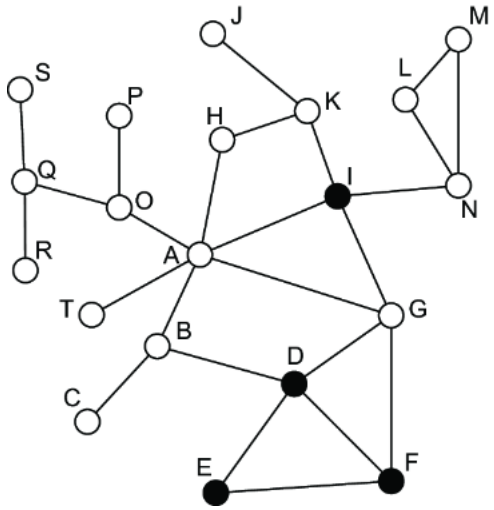

Figure 10.9 Our simple fraudulent network. In this case, the colors of the nodes will be ignored, because ReFeX doesn't consider node types.

Table 10.9 Degree of the nodes in figure 10.9

Node	Degree
A	6
B	3
C	1
D	4
E	2
…	…

These results are the same as in section 10.1.1. This is not a directed graph, so we can't compute the in-degree and out-degree.

The next step is to consider the egonet for each node. For example, for node A, the egonet is composed of A itself and its neighbors: B, G, H, I, O, and T. The total number of nodes in the egonet is seven (A plus its six neighbors). The total number of internal edges is seven (six edges connecting A to its neighbors, plus one edge between G and I). The total number of in/out egonet edges is nine (these are edges that connect nodes in A's egonet to nodes outside: B→C,D; G→D,F; H→K; I→K,N; O→P,Q). Table 10.10 lists the total values for each of the nodes.

Table 10.10 Details of the egonet structure for the nodes in figure 10.9

Node's egonet	# of nodes	# of internal edges	# of in/out edges
A	7	7	9
B	4	3	5
C	2	1	1
D	5	6	4
E	3	3	3
...	...	...	...

The final step in ReFeX involves recursive feature aggregation, a multi-phase process that progressively captures broader network characteristics. The algorithm aggregates features from increasingly distant neighborhoods at each iteration, creating a more comprehensive view of each node's structural context. Figure 10.10 illustrates this process using our example graph.

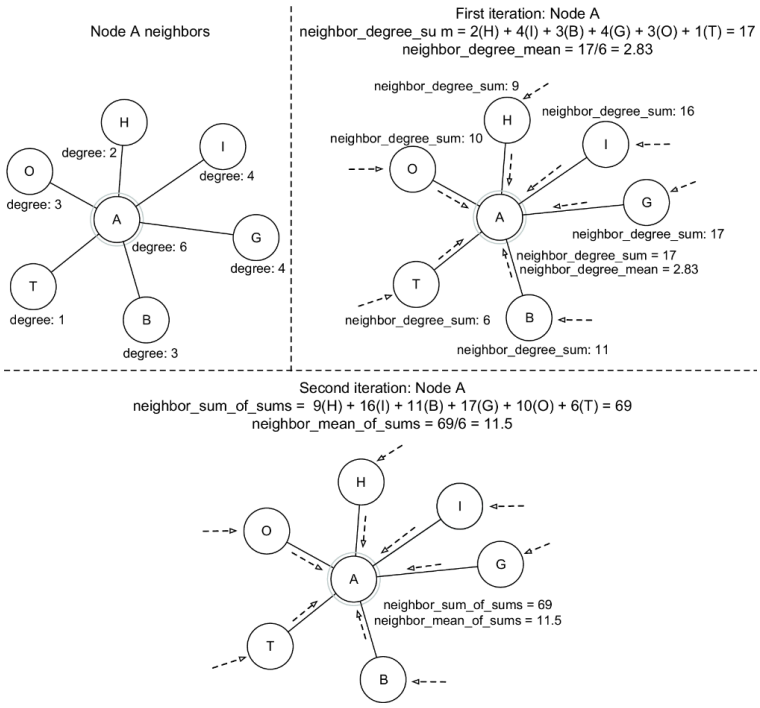

Figure 10.10 Illustration of a few iterations of the ReFeX process. At each iteration, each node passes its values (degree, previous sums, etc.) to the neighbors, which will aggregate.

Let's walk through the process using node A as our example:

1. First iteration:
    1. Node A has six neighbors (H, I, G, B, T, O).
    2. Local features: degree(A) = 6
    3. First aggregation using the `SUM` operator:

$$\begin{aligned}\text{sum}(\text{neighbor\_degrees}) &= \text{degree}(H) + \text{degree}(I) + \text{degree}(G) + \text{degree}(B) \\ &\quad + \text{degree}(T) + \text{degree}(O) \\ \text{sum}(\text{neighbor\_degrees}) &= 2 + 4 + 4 + 3 + 1 + 3 = 17\end{aligned}$$

21. Second iteration:
    1. Uses the aggregated values from the first iteration.
    2. Each neighbor now carries its first-iteration sum (agg).

3. Second aggregation:

$$\text{sum(neighbor\_aggregates)} = \text{agg}(B) + \text{agg}(G) + \text{agg}(H) + \text{agg}(I) + \text{agg}(O) + \text{agg}(T)$$
$$\text{sum(neighbor\_aggregates)} = 11 + 17 + 9 + 16 + 10 + 6 = 69$$

Additionally, ReFeX can use the `MEAN` operator for aggregation. For node A, this would give

- First iteration:

mean(neighbor_degrees) = (3 + 4 + 2 + 4 + 3 + 1)/6 = 17/6 ≈ 2.83

- Second iteration:

mean(neighbor_aggregates) = (11 + 17 + 9 + 16 + 10 + 6)/6 = 69/6 = 11.5

Table 10.11 shows the results for node A after these iterations (before pruning).

Table 10.11 Features computed for node A

Feature type	Feature	Value
Local feature	Degree	6
Egonet feature	Number of edges in the egonet	7
Recursive feature, first iteration	Sum of neighbor degrees	17
	Mean of neighbor degrees	2.83
Recursive feature, second iteration	Sum of neighbor sums	69
	Mean of neighbor sums	11.5

10.3.2 Performing ReFeX automatically with code

The full implementation of the algorithm is available in the book's code repository. The following listing shows the most relevant part of the code.

Listing 10.18 ReFeX implementation (key parts)

```
import networkx as nx
import numpy as np
from collections import defaultdict
from sklearn.preprocessing import StandardScaler

class ReFeX:
    def __init__(self, max_iterations=2, correlation_threshold=0.95):
        self.max_iterations = max_iterations
        self.correlation_threshold = correlation_threshold   #1

    def extract_features(self, G):
        features = self._extract_local_features(G)   #2

        egonet_features = self._extract_egonet_features(G)
        features = np.column_stack((features, egonet_features))   #3

        for iteration in range(self.max_iterations):
            new_features = self._generate_recursive_features(G, features)
            features = np.column_stack((features, new_features))   #4
            features = self._prune_features(features)   #5
        return features

    def _extract_local_features(self, G):
        """Extract local (node-level) features"""
        n_nodes = G.number_of_nodes()
        features = np.zeros((n_nodes, 3))

        for idx, node in enumerate(G.nodes()):
            # Degree features
            features[idx, 0] = G.degree(node)

            # In-degree and out-degree for directed graphs
            if G.is_directed():
                features[idx, 1] = G.in_degree(node)
                features[idx, 2] = G.out_degree(node)
            else:
                features[idx, 1] = features[idx, 2] = G.degree(node)   #6

        return features

    def _extract_egonet_features(self, G):  #7
        n_nodes = G.number_of_nodes()
        features = np.zeros((n_nodes, 3))
        for idx, node in enumerate(G.nodes()):
            ego = nx.ego_graph(G, node, radius=1)
            features[idx, 0] = ego.number_of_nodes()
            features[idx, 1] = ego.number_of_edges()
            features[idx, 2] = nx.density(ego)

        return features

    def _generate_recursive_features(self, G, current_features):   #8
        n_nodes = G.number_of_nodes()
        n_features = current_features.shape[1]
        new_features = np.zeros((n_nodes, n_features * 2))

        for idx, node in enumerate(G.nodes()):
            neighbors = list(G.neighbors(node))
            if not neighbors:
                continue

            neighbor_feats =
               current_features[[list(G.nodes()).index(n) for n in neighbors]]
               new_features[idx, :n_features] = np.sum(neighbor_feats, axis=0)
               new_features[idx, n_features:] = np.mean(neighbor_feats,
               axis=0)
```

```
        return new_features

    def _prune_features(self, features):   #9
        scaler = StandardScaler()
        scaled_features = scaler.fit_transform(features)   #10

        corr_matrix = np.corrcoef(scaled_features.T)   #11

        to_remove = set()
        for i in range(corr_matrix.shape[0]):   #12
            for j in range(i + 1, corr_matrix.shape[1]):
                if abs(corr_matrix[i, j]) > self.correlation_threshold:
                    to_remove.add(j)

        keep_features = list(set(range(features.shape[1])) - to_remove)   #13
        return features[:, keep_features]   #14
```

#1 Initializes ReFeX with an iteration limit and correlation threshold
#2 Extracts basic node-level features (degrees) and uses them to initialize the feature matrix
#3 Adds features based on egonet properties
#4 Generates and adds recursive features iteratively
#5 Removes redundant features based on correlation
#6 Computes local features including degree metrics
#7 Calculates egonet-based features for each node
#8 Generates recursive features using sum and mean aggregations
#9 Identifies highly correlated features for removal, and removes them
#10 Standardizes features
#11 Computes the correlation matrix
#12 Finds highly correlated features
#13 Keeps uncorrelated features
#14 Returns a pruned feature matrix

EXERCISE

The book's code repository contains the complete code, which connects to a Hetionet database available in Neo4j. Give it a try.

ReFeX represents a significant step toward automated feature extraction, occupying an important middle ground between manual feature engineering and fully autonomous representation learning techniques. Its focus on pure graph structure provides an excellent foundation for understanding more complex approaches and demonstrates how structural patterns alone can capture meaningful characteristics of nodes and their neighborhoods. Although ReFeX automates feature extraction, it maintains transparency and interpretability; its computations can be traced and verified step by step, making it an invaluable tool for practitioners who need to understand and validate their feature engineering process. And ReFeX's deterministic nature ensures consistency: identical inputs will always produce identical outputs. This predictability is particularly valuable in production environments where reproducibility is essential. Moreover, when the graph structure changes, ReFeX allows for selective recomputation of affected features rather than requiring a complete regeneration of the feature matrix. This efficiency makes it particularly suitable for dynamic graph environments.

However, ReFeX also has limitations. Its reliance on structural features means it cannot directly incorporate node attributes or edge types. And although pruning helps manage computational complexity, it sometimes requires human oversight to ensure optimal feature selection.

In the next chapter, we'll see how modern autonomous representation learning techniques address these limitations while sacrificing some of the interpretability and deterministic nature that make ReFeX valuable for certain applications. Understanding ReFeX's strengths and limitations

will provide essential context for appreciating the innovations and trade-offs introduced by these more advanced methods.

Summary

- Manual and semimanual feature engineering in graphs provides a foundation for ML tasks, balancing interpretability with automation.
- Manual feature engineering combines local metrics with global measures to create meaningful node representations that capture both immediate connections and broader network patterns.
- Combining node metrics helps identify patterns while maintaining interpretable decision-making processes.
- Relationship representation can be approached through node-based combinations (concatenation, averaging, distances) or path-based features (metapaths, structural patterns).
- Node-based relationship representation can use various combination methods: concatenation, averaging, L1/L2 distances, and Hadamard product.
- Path-based features capture structural patterns between nodes through metapath analysis.
- DWPC (degree-weighted path count) provides a sophisticated approach to measuring connection relevance while accounting for node degrees.
- Domain expertise guides feature engineering in both node and relationship representation, from metric selection to validation of results.
- Semiautomated approaches like ReFeX offer a middle ground by automatically generating interpretable features while preserving the option to incorporate domain knowledge.
- The choice between manual and semiautomated approaches depends on interpretability needs, computational resources, and available domain expertise.
- Feature pruning and selection remain essential in both approaches, requiring careful consideration of correlation and relevance to specific ML tasks.